

Assignment 2 (Extension of PE5 and PE6)

Krishank Kureti

PES2UG23CS285

Sem 6 'E'

PE-5

abstract_syntax_tree.h

```
#ifndef ABSTRACT_SYNTAX_TREE_H
#define ABSTRACT_SYNTAX_TREE_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct expression_node {
    char* val;
    struct expression_node* left;
    struct expression_node* right;
} expression_node;

typedef enum { LT_OP, GT_OP, LE_OP, GE_OP, EQ_OP, NE_OP } relational_operator;

typedef struct conditional_node {
    char* left_id;
    relational_operator op;
    char* right_id;
} conditional_node;

typedef struct statement_node {
    enum { ASSIGN_STMT, IF_ELSE_STMT, DO_WHILE_STMT } type;
    union {
        struct { char* id; expression_node* exp; } assign;
        struct if_else_statement_node* if_else_stmt;
        struct do_while_statement_node* do_while_stmt;
    } stmt;
    struct statement_node* next;
}
```

```

} statement_node;

typedef struct if_else_statement_node {
    conditional_node* condition;
    statement_node* true_branch;
    statement_node* false_branch;
} if_else_statement_node;

typedef struct do_while_statement_node {
    statement_node* loop_body;
    conditional_node* condition;
} do_while_statement_node;

expression_node* init_exp_node(char* val, expression_node* left, expression_node* right);
conditional_node* init_conditional_node(char* left_id, relational_operator op, char* right_id);
statement_node* init_assign_stmt(char* id, expression_node* exp, statement_node* next);
statement_node* init_if_else_stmt(conditional_node* cond, statement_node* t, statement_node*
f, statement_node* next);
statement_node* init_do_while_stmt(statement_node* body, conditional_node* cond,
statement_node* next);
void display_ast(statement_node* root);

#endif

```

abstract_syntax_tree.c

```

#include "abstract_syntax_tree.h"

expression_node* init_exp_node(char* val, expression_node* left, expression_node* right) {
    expression_node* node = malloc(sizeof(expression_node));
    node->val = strdup(val);
    node->left = left;
    node->right = right;
    return node;
}

conditional_node* init_conditional_node(char* left_id, relational_operator op, char* right_id) {
    conditional_node* node = malloc(sizeof(conditional_node));
    node->left_id = strdup(left_id);
    node->op = op;
    node->right_id = strdup(right_id);
    return node;
}

```

```

statement_node* init_assign_stmt(char* id, expression_node* exp, statement_node* next) {
    statement_node* node = malloc(sizeof(statement_node));
    node->type = ASSIGN_STMT;
    node->stmt.assign.id = strdup(id);
    node->stmt.assign.exp = exp;
    node->next = next;
    return node;
}

```

```

statement_node* init_if_else_stmt(conditional_node* cond, statement_node* t, statement_node*
f, statement_node* next) {
    statement_node* node = malloc(sizeof(statement_node));
    node->type = IF_ELSE_STMT;
    node->stmt.if_else_stmt = malloc(sizeof(if_else_statement_node));
    node->stmt.if_else_stmt->condition = cond;
    node->stmt.if_else_stmt->true_branch = t;
    node->stmt.if_else_stmt->>false_branch = f;
    node->next = next;
    return node;
}

```

```

statement_node* init_do_while_stmt(statement_node* body, conditional_node* cond,
statement_node* next) {
    statement_node* node = malloc(sizeof(statement_node));
    node->type = DO_WHILE_STMT;
    node->stmt.do_while_stmt = malloc(sizeof(do_while_statement_node));
    node->stmt.do_while_stmt->loop_body = body;
    node->stmt.do_while_stmt->condition = cond;
    node->next = next;
    return node;
}

```

```

const char* get_op_str(relational_operator op) {
    switch(op) {
        case LT_OP: return "<"; case GT_OP: return ">";
        case LE_OP: return "<="; case GE_OP: return ">=";
        case EQ_OP: return "=="; case NE_OP: return "!=";
        default: return "?";
    }
}

```

```

// Helper: Pre-order traversal for expressions
void display_exp_preorder(expression_node* node) {
    if (!node) return;

```

```

    printf("%s\n", node->val);
    display_exp_preorder(node->left);
    display_exp_preorder(node->right);
}

void display_stmt(statement_node* node) {
    while (node) {
        if (node->type == ASSIGN_STMT) {
            printf("%s\n", node->stmt.assign.id);
            display_exp_preorder(node->stmt.assign.exp);
        } else if (node->type == IF_ELSE_STMT) {
            printf("if\n");
            printf("%s\n%s\n%s\n", node->stmt.if_else_stmt->condition->left_id,
                get_op_str(node->stmt.if_else_stmt->condition->op),
                node->stmt.if_else_stmt->condition->right_id);
            display_stmt(node->stmt.if_else_stmt->true_branch);
            printf("else\n");
            display_stmt(node->stmt.if_else_stmt->>false_branch);
        } else if (node->type == DO_WHILE_STMT) {
            printf("do\n");
            display_stmt(node->stmt.do_while_stmt->loop_body);
            printf("while\n");
            printf("%s\n%s\n%s\n", node->stmt.do_while_stmt->condition->left_id,
                get_op_str(node->stmt.do_while_stmt->condition->op),
                node->stmt.do_while_stmt->condition->right_id);
        }
        node = node->next;
    }
}

void display_ast(statement_node* root) {
    display_stmt(root);
}

```

parser.y

```

%{
#include "abstract_syntax_tree.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void yyerror(const char* s);
int yylex();

```

```
extern int yylineno;
%}
```

```
%union {
    char* text;
    struct expression_node* exp_node;
    struct statement_node* stmt_node;
    struct conditional_node* cond_node;
    int rel_op;
}
```

```
%token <text> T_ID T_NUM
%token IF ELSE DO WHILE
%token LT GT LE GE EQ NE
%type <exp_node> E T F
%type <cond_node> C
%type <rel_op> rel
%type <stmt_node> S
```

```
%start START
```

```
%%
```

```
START : S {
        if ($1 != NULL) display_ast($1);
        printf("Valid syntax\n");
        YYACCEPT;
    }
;
```

```
S : T_ID '=' E S { $$ = init_assign_stmt($1, $3, $4); }
  | IF '(' C ')' '{' S '}' ELSE '{' S '}' S { $$ = init_if_else_stmt($3, $6, $10, $12); }
  | DO '{' S '}' WHILE '(' C ')' ';' S { $$ = init_do_while_stmt($3, $7, $10); }
  | /* epsilon */ { $$ = NULL; }
;
```

```
C : T_ID rel T_ID { $$ = init_conditional_node($1, $2, $3); } ;
rel : LT { $$ = LT_OP; } | GT { $$ = GT_OP; } | LE { $$ = LE_OP; } | GE { $$ = GE_OP; } | EQ {
    $$ = EQ_OP; } | NE { $$ = NE_OP; } ;
E : E '+' T { $$ = init_exp_node("+", $1, $3); } | E '-' T { $$ = init_exp_node("-", $1, $3); } | T { $$
    = $1; } ;
T : T '*' F { $$ = init_exp_node("*", $1, $3); } | T '/' F { $$ = init_exp_node("/", $1, $3); } | F { $$ =
    $1; } ;
```

```
F : '(' E ')' { $$ = $2; } | T_ID { $$ = init_exp_node($1, NULL, NULL); } | T_NUM { $$ =
init_exp_node($1, NULL, NULL); } ;
```

```
%%
```

```
void yyerror(const char* s) { printf("Error: %s at line %d\n", s, yylineno); }
```

```
int main() { yyparse(); return 0; }
```

lexer.l

```
%{
```

```
#include "abstract_syntax_tree.h"
```

```
#include "y.tab.h"
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
%}
```

```
%option yylineno
```

```
%%
```

```
"if"      { return IF; }
```

```
"else"    { return ELSE; }
```

```
"do"      { return DO; }
```

```
"while"   { return WHILE; }
```

```
"<="      { return LE; }
```

```
">="      { return GE; }
```

```
"=="      { return EQ; }
```

```
"!="      { return NE; }
```

```
"<"       { return LT; }
```

```
">"       { return GT; }
```

```
"="        { return '='; }
```

```
"+"        { return '+'; }
```

```
"-"        { return '-'; }
```

```
"*"        { return '*'; }
```

```
"/"        { return '/'; }
```

```
"("        { return '('; }
```

```
")"        { return ')'; }
```

```
"{"        { return '{'; }
```

```
"}"        { return '}'; }
```

```
","        { return ','; }
```

```
[0-9]+(\\.[0-9]+)? { yylval.text = strdup(yytext); return T_NUM; }
```

```
[a-zA-Z_][a-zA-Z0-9_]* { yylval.text = strdup(yytext); return T_ID; }
```

```
[ \\t\\n] ;
```

```
. { return *yytext; }
```

%%

int yywrap() { return 1; }

test_case_1.c

$a = 10 / 5 - 2 * 7 + 3$

test_case_2.c

$b = c / 6.7 + 12.45 - a * 1234.0$

test_case_3.c

```
if (a < b) {  
    x = 10 + 5  
} else {  
    x = 20 - 5  
}  
do {  
    y = y + 1  
} while (y < z);
```

Output

```
[krishankkureti@Krishanks-MacBook-Air CD-PE5 % bash run.sh
```

```
=====
```

Test Case 1:

a

+

-

/

10

5

*

2

7

3

Valid syntax

```
-----
```

Test Case 2:

b

-

+

/

c

6.7

12.45

*

a

1234.0

Valid syntax

```
-----
```

Test Case 3:

if

a

<

b

x

+

10

5

else

x

-

20

5

do

y

+

y

1

while

y

<

z

Valid syntax

```
=====
```


PE-6

quad_generation.h

```
#ifndef QUAD_GENERATION_H
#define QUAD_GENERATION_H
#include <stdio.h>
extern FILE* icg_quad_file;
extern int temp_no;
extern int label_no;
void quad_code_gen(char* op, char* b, char* c, char* a);
char* new_temp();
char* new_label();
#endif
```

quad_generation.c

```
#include "quad_generation.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int label_no = 1;

void quad_code_gen(char* op, char* b, char* c, char* a) {
    fprintf(icg_quad_file, "%s, %s, %s, %s\n", op, b, c, a);
}

char* new_temp() {
    char* temp = (char*)malloc(10);
    sprintf(temp, "t%d", temp_no++);
    return temp;
}

char* new_label() {
    char* label = (char*)malloc(10);
    sprintf(label, "L%d", label_no++);
    return label;
}
```

parser.y

```
%{
    #include "quad_generation.h"
    #include <stdio.h>
    #include <stdlib.h>
    #include <string.h>

    void yyerror(char* s);
    int yylex();
    extern int yylineno;
    FILE* icg_quad_file;
    int temp_no = 1;
}%}

%union {
    char* text;
}

%token <text> T_ID T_NUM IF ELSE DO WHILE
%token <text> LT GT LE GE EQ NE

/* S and START are now untyped to stop the 'type clash' warnings */
%type <text> E T F C rel

%start START

%%

START : S { printf("Valid syntax\n"); YYACCEPT; }
      ;

S : T_ID '=' E S {
    quad_code_gen("=", $3, "", $1);
}
| IF '(' C ')' '{' {
    /* Marker for If: Jump to Else if False */
    $<text>$ = new_label();
    quad_code_gen("ifFalse", $3, "", $<text>$);
} S '}' ELSE '{' {
    /* Marker for Else: Jump to Next; Place Else Label */
    char* l_next = new_label();
    quad_code_gen("goto", "", "", l_next);
    quad_code_gen("Label", "", "", $<text>6);
}
```

```

    $<text>$ = l_next;
} S '}' {
    /* Place Next Label */
    quad_code_gen("Label", "", "", $<text>11);
} S
| DO '{' {
    /* Marker for Do: Place Start Label */
    $<text>$ = new_label();
    quad_code_gen("Label", "", "", $<text>$);
} S '}' WHILE '(' C ')' ';' {
    /* Jump back to Start if True */
    quad_code_gen("ifTrue", $8, "", $<text>3);
} S
| /* epsilon */
;

C : T_ID rel T_ID {
    $$ = new_temp();
    quad_code_gen($2, $1, $3, $$);
}
;

rel : LT { $$ = "<"; } | GT { $$ = ">"; } | LE { $$ = "<="; }
    | GE { $$ = ">="; } | EQ { $$ = "=="; } | NE { $$ = "!="; }
;

E : E '+' T { $$ = new_temp(); quad_code_gen("+", $1, $3, $$); }
    | E '-' T { $$ = new_temp(); quad_code_gen("-", $1, $3, $$); }
    | T { $$ = $1; }
;

T : T '*' F { $$ = new_temp(); quad_code_gen("*", $1, $3, $$); }
    | T '/' F { $$ = new_temp(); quad_code_gen("/", $1, $3, $$); }
    | F { $$ = $1; }
;

F : '(' E ')' { $$ = $2; }
    | T_ID { $$ = strdup($1); }
    | T_NUM { $$ = strdup($1); }
;

%%

void yyerror(char* s) { printf("Error: %s at %d\n", s, yylineno); }

```

```

int main() {
    icg_quad_file = fopen("icg_quad.txt", "w");
    yyparse();
    fclose(icg_quad_file);
    return 0;
}

```

lexer.l

```

%{
    #include "y.tab.h"
    #include <stdio.h>
    #include <string.h>
}%

%option yylineno

%%

"if"      { return IF; }
"else"    { return ELSE; }
"do"      { return DO; }
"while"   { return WHILE; }
"<="      { return LE; }
">="      { return GE; }
"=="      { return EQ; }
"!="      { return NE; }
"<"       { return LT; }
">"       { return GT; }
[0-9]+(\\.[0-9]+)? { yylval.text = strdup(yytext); return T_NUM; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.text = strdup(yytext); return T_ID; }
"//".* ;
[ \\t\\n] ;
.         { return *yytext; }
%%

int yywrap() { return 1; }

```

test_case_1.c

$x = 9/2 + a - b$

test_case_2.c

```
b = c / 6.7 + 12.45 - a * 1234.0
```

test_case_3.c

```
if (a < b) {  
    x = 10 + 5  
} else {  
    x = 20 - 5  
}  
do {  
    y = y + 1  
} while (y < z);
```

Output

```
[krishankkureti@Krishanks-MacBook-Air CD-PE6 % bash run.sh
```

```
=====
```

```
Test Case 1 (Sample ICG):
```

```
Valid syntax
```

```
/, 9, 2, t1
```

```
+, t1, a, t2
```

```
-, t2, b, t3
```

```
=, t3, , x
```

```
-----
```

```
Test Case 2 (Sample ICG):
```

```
Valid syntax
```

```
/, c, 6.7, t1
```

```
+, t1, 12.45, t2
```

```
*, a, 1234.0, t3
```

```
-, t2, t3, t4
```

```
=, t4, , b
```

```
-----
```

```
Test Case 3 (Control Flow ICG):
```

```
Valid syntax
```

```
<, a, b, t1
```

```
ifFalse, t1, , L1
```

```
+, 10, 5, t2
```

```
=, t2, , x
```

```
goto, , , L2
```

```
Label, , , L1
```

```
-, 20, 5, t3
```

```
=, t3, , x
```

```
Label, , , L2
```

```
Label, , , L3
```

```
+, y, 1, t4
```

```
=, t4, , y
```

```
<, y, z, t5
```

```
ifTrue, t5, , L3
```

```
=====
```